AI ASSISTED CODING ASS-12.3

2303A51878

B-28

Task 1: Sorting Student Records for Placement Drive

Scenario: SR University's Training and Placement Cell need to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks:

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

- Quick Sort
- Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

CODE:

```
import time
import random

class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [s for s in students if s.cgpa > pivot]
    middle = [s for s in students if s.cgpa == pivot]
    right = [s for s in students if s.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

```python
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])

    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]
```

```python
# Main execution
if __name__ == "__main__":
    students = [Student(f"Student_{i}", 1001 + i, round(random.uniform(5.0, 10.0), 2))
                for i in range(1000)]

    start = time.time()
    sorted_quick = quick_sort(students.copy())
    quick_time = time.time() - start

    start = time.time()
    sorted_merge = merge_sort(students.copy())
    merge_time = time.time() - start

    print("\n=== Top 10 Students ===")
    for i, s in enumerate(sorted_quick[:10], 1):
        print(f"{i}. {s.name} (Roll: {s.roll_number}) - CGPA: {s.cgpa}")

    print(f"\nQuick Sort: {quick_time:.6f}s | Merge Sort: {merge_time:.6f}s")
```

Output:

```
=== Top 10 Students ===
1. Student_271 (Roll: 1272) - CGPA: 10.0
2. Student_168 (Roll: 1169) - CGPA: 9.99
3. Student_730 (Roll: 1731) - CGPA: 9.99
4. Student_615 (Roll: 1616) - CGPA: 9.98
5. Student_846 (Roll: 1847) - CGPA: 9.98
6. Student_148 (Roll: 1149) - CGPA: 9.97
7. Student_477 (Roll: 1478) - CGPA: 9.97
8. Student_534 (Roll: 1535) - CGPA: 9.97
9. Student_739 (Roll: 1740) - CGPA: 9.97
10. Student_761 (Roll: 1762) - CGPA: 9.96

Quick Sort: 0.001152s | Merge Sort: 0.001533s
```

Task 2: Implementing Bubble Sort with AI Comments

Task: Write a Python implementation of Bubble Sort.

Instructions:

- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis.

CODE:

```python
def bubble_sort(students):
    """
    Bubble Sort Algorithm - Sorts students by CGPA in descending order.

    Time Complexity: O(n²) in average and worst case, O(n) in best case
    Space Complexity: O(1) - sorts in place
    """
    n = len(students)

    # Outer loop controls the number of passes through the array
    # Each pass moves the largest unsorted element to its correct position
    for i in range(n):
        # Flag to optimize: if no swaps occur in a pass, array is already sorted
        swapped = False

        # Inner loop compares adjacent elements and swaps if needed
        # We reduce the range each iteration because the last i elements are already in place
        for j in range(0, n - i - 1):
            # Compare CGPA values of adjacent students
            # If left student has lower CGPA than right, swap them (descending order)
            if students[j].cgpa < students[j + 1].cgpa:
                # Swap operation: exchange positions of two students
                students[j], students[j + 1] = students[j + 1], students[j]
                swapped = True

        # Termination optimization: if no swaps occurred, array is sorted
        # This reduces time complexity to O(n) in best case (already sorted array)
        if not swapped:
            break

    return students
```

```
# Time Complexity Analysis:
# ========================
# Best Case: O(n)
#   - Occurs when the array is already sorted
#   - Only one pass needed; no swaps occur
#   - The 'swapped' flag optimization prevents unnecessary iterations
#
# Average Case: O(n²)
#   - Typical scenario with random data
#   - Multiple passes required with several comparisons and swaps
#   - First pass makes n-1 comparisons, second makes n-2, etc.
#   - Total: (n-1) + (n-2) + ... + 1 = n(n-1)/2 ≈ O(n²)
#
# Worst Case: O(n²)
#   - Occurs when array is sorted in reverse order (opposite of desired output)
#   - Every comparison requires a swap
#   - Maximum number of passes and swaps needed
#   - Still n(n-1)/2 comparisons total
#
# Space Complexity: O(1)
#   - Uses only a constant amount of extra memory
#   - Sorting happens in-place with no additional data structures
#   - Only uses a few variables (i, j, swapped, temp storage for swap)
```

```
# Example Usage and Testing
if __name__ == "__main__":
    import time
    import random

    class Student:
        def __init__(self, name, roll_number, cgpa):
            self.name = name
            self.roll_number = roll_number
            self.cgpa = cgpa

        def __repr__(self):
            return f"{self.name}({self.cgpa})"

    # Generate test data
    students = [Student(f"Student_{i}", 1001 + i, round(random.uniform(5.0, 10.0), 2))
                for i in range(100)]

    print("=== Bubble Sort Performance Test ===\n")
    print("Before sorting (first 5):")
    for s in students[:5]:
        print(f"  {s.name} - CGPA: {s.cgpa}")

    # Measure performance
    start = time.time()
    sorted_students = bubble_sort(students.copy())
    bubble_time = time.time() - start

    print(f"\nAfter sorting (top 5):")
    for i, s in enumerate(sorted_students[:5], 1):
        print(f"  {i}. {s.name} - CGPA: {s.cgpa}")

    print(f"\nBubble Sort Time: {bubble_time:.6f} seconds")
    print(f"Array Size: {len(students)} students")
```

Output:

```
=== Bubble Sort Performance Test ===

Before sorting (first 5):
  Student_0 - CGPA: 6.94
  Student_1 - CGPA: 5.95
  Student_2 - CGPA: 7.26
  Student_3 - CGPA: 8.37
  Student_4 - CGPA: 6.47

After sorting (top 5):
  1. Student_91 - CGPA: 9.94
  2. Student_33 - CGPA: 9.93
  3. Student_50 - CGPA: 9.83
  4. Student_82 - CGPA: 9.79
  5. Student_72 - CGPA: 9.72

Bubble Sort Time: 0.000277 seconds
Array Size: 100 students
```

Task 3: Quick Sort and Merge Sort Comparison

Task: Implement Quick Sort and Merge Sort using recursion.

Instructions:

- Provide AI with partially completed functions for recursion.
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted lists.

CODE:

```python
import time
import random
from copy import deepcopy

class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.name}({self.cgpa})"

# Quick Sort
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [s for s in students if s.cgpa > pivot]
    middle = [s for s in students if s.cgpa == pivot]
    right = [s for s in students if s.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

```python
# Merge Sort
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])

    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def generate_test_data(n):
    return [Student(f"Student_{i}", 1001 + i, round(random.uniform(5.0, 10.0), 2)) for i in range(n)]

def compare_algorithms(n=1000):
    data = generate_test_data(n)

    print(f"\nComparing algorithms on {n} students\n")

    start = time.time()
    quick_sort(deepcopy(data))
    print(f"Quick Sort: {time.time() - start:.6f} seconds")

    start = time.time()
    merge_sort(deepcopy(data))
    print(f"Merge Sort: {time.time() - start:.6f} seconds")

if __name__ == "__main__":
    compare_algorithms(n=1000)
```

Output:

```
Comparing algorithms on 1000 students

Quick Sort: 0.005546 seconds
Merge Sort: 0.005046 seconds
```

Explanation:

Best-case complexity

This is the minimum time an algorithm takes for any valid input of size *n*.
It happens when the input is in the most favourable condition for the algorithm.
Example: Linear search when the element is found at the first position → **O(1)**.

Average-case complexity

This represents the expected time an algorithm takes over all possible inputs of size *n*.
It assumes that all inputs are equally likely and gives a realistic performance measure.
Example: Linear search where the element is found somewhere in the middle → **O(n)**.

Worst-case complexity

This is the maximum time an algorithm can take for any input of size *n*.
It occurs when the input is in the least favourable condition and is used for performance guarantees.
Example: Linear search when the element is not present or at the last position → **O(n)**.

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products,

each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

CODE:

```
class Product:
    def __init__(self, pid, name, price, quantity):
        self.pid = pid
        self.name = name
        self.price = price
        self.quantity = quantity

    def __str__(self):
        return f"ID: {self.pid}, Name: {self.name}, Price: {self.price}, Quantity: {self.quantity}"

inventory = []
id_index = {}
name_index = {}

def add_product(pid, name, price, quantity):
    product = Product(pid, name, price, quantity)
    inventory.append(product)
    id_index[pid] = product
    name_index[name.lower()] = product

def search_by_id(pid):
    return id_index.get(pid)

def search_by_name(name):
    return name_index.get(name.lower())

def sort_by_price():
    return sorted(inventory, key=lambda x: x.price)

def sort_by_quantity():
    return sorted(inventory, key=lambda x: x.quantity)
```

```
add_product(101, "Laptop", 60000, 15)
add_product(102, "Mouse", 500, 120)
add_product(103, "Keyboard", 1500, 60)
add_product(104, "Monitor", 12000, 25)

print("Search by Product ID (102):")
print(search_by_id(102))

print("\nSearch by Product Name (Laptop):")
print(search_by_name("Laptop"))

print("\nProducts Sorted by Price:")
for p in sort_by_price():
    print(p)

print("\nProducts Sorted by Quantity:")
for p in sort_by_quantity():
    print(p)
```

Output:

```
Search by Product ID (102):
ID: 102, Name: Mouse, Price: 500, Quantity: 120

Search by Product Name (Laptop):
ID: 101, Name: Laptop, Price: 60000, Quantity: 15

Products Sorted by Price:
ID: 102, Name: Mouse, Price: 500, Quantity: 120
ID: 103, Name: Keyboard, Price: 1500, Quantity: 60
ID: 104, Name: Monitor, Price: 12000, Quantity: 25
ID: 101, Name: Laptop, Price: 60000, Quantity: 15

Products Sorted by Quantity:
ID: 101, Name: Laptop, Price: 60000, Quantity: 15
ID: 104, Name: Monitor, Price: 12000, Quantity: 25
ID: 103, Name: Keyboard, Price: 1500, Quantity: 60
ID: 102, Name: Mouse, Price: 500, Quantity: 120
```

Algorithm Mapping Table:

Operation	**Recommended Algorithm**	**Justification**
Search by Product ID	Hash Table (Dictionary)	O(1) average time complexity. Ideal for frequent exact searches. Product IDs are unique, making hashing perfect.
Search by Product Name	Hash Table (Dictionary)	Fast lookup for exact name matches. Avoids linear scans over thousands of items.
Sort by Price	Timsort (Python's built-in sort)	O(n log n) worst case, optimized for real-world data. Stable and fast for large datasets.
Sort by Quantity	Timsort (Python's built-in sort)	Same benefits as price sorting. Efficient and reliable for analytics.

Task 5: Real-Time Stock Data Sorting & Searching

Scenario: An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

CODE:

```
class Stock:
    def __init__(self, symbol, open_price, close_price):
        self.symbol = symbol
        self.open = open_price
        self.close = close_price
        self.change = ((close_price - open_price) / open_price) * 100

    def __str__(self):
        return f"{self.symbol} | Open: {self.open} | Close: {self.close} | Change: {self.change:.2f}%"

stocks = [
    Stock("AAPL", 180, 189),
    Stock("GOOG", 2700, 2680),
    Stock("TSLA", 250, 275),
    Stock("AMZN", 135, 130),
    Stock("MSFT", 310, 325)
]

stock_map = {}
for s in stocks:
    stock_map[s.symbol] = s

def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2

    if l < n and arr[l].change > arr[largest].change:
        largest = l
    if r < n and arr[r].change > arr[largest].change:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

```
def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
    return arr

def search_stock(symbol):
    return stock_map.get(symbol)

print("Search Result for TSLA:")
print(search_stock("TSLA"))

print("\nStocks Sorted by % Change using Heap Sort:")
heap_sorted = heap_sort(stocks.copy())
for s in heap_sorted[::-1]:
    print(s)

print("\nStocks Sorted by % Change using sorted():")
lib_sorted = sorted(stocks, key=lambda x: x.change, reverse=True)
for s in lib_sorted:
    print(s)
```

Output:

```
Search Result for TSLA:
TSLA | Open: 250 | Close: 275 | Change: 10.00%

Stocks Sorted by % Change using Heap Sort:
TSLA | Open: 250 | Close: 275 | Change: 10.00%
AAPL | Open: 180 | Close: 189 | Change: 5.00%
MSFT | Open: 310 | Close: 325 | Change: 4.84%
GOOG | Open: 2700 | Close: 2680 | Change: -0.74%
AMZN | Open: 135 | Close: 130 | Change: -3.70%

Stocks Sorted by % Change using sorted():
TSLA | Open: 250 | Close: 275 | Change: 10.00%
AAPL | Open: 180 | Close: 189 | Change: 5.00%
MSFT | Open: 310 | Close: 325 | Change: 4.84%
GOOG | Open: 2700 | Close: 2680 | Change: -0.74%
AMZN | Open: 135 | Close: 130 | Change: -3.70%
```